Cloud Kubernetes Penetration testing and Configuration Review Setup

What will we be using?

1. MiniKube – Minikube creates a single-node Kubernetes cluster locally, ideal for hosting Kubernetes Goat on laptops with limited resources. It requires 2 CPUs, 2GB RAM, 20GB disk, and a container runtime like Docker.
2. Kubernetes Goat - Kubernetes Goat is a deliberately vulnerable Kubernetes cluster designed as an interactive playground for learning and practicing Kubernetes security. It features multiple scenarios based on real-world misconfigurations, such as exposed secrets and container escapes, to help users understand attack vectors and defenses. It simulates common issues in cloud-native environments without relying on outdated software vulnerabilities

1. **Setting up MiniKube in your local linux machine.**

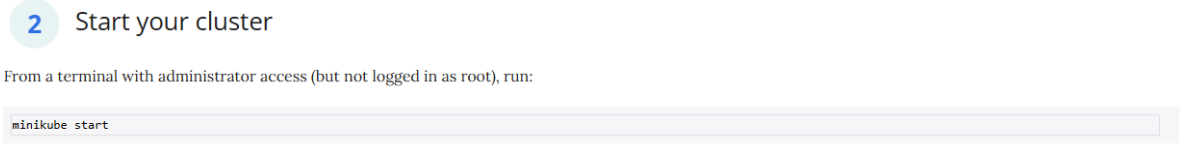
Minikube is local Kubernetes, focusing on making it easy to learn and develop for Kubernetes.

Ref : - https://minikube.sigs.k8s.io/docs/start/?arch=%2Fwindows%2Fx86-64%2Fstable%2F.exe+download

Steps to perform-



1. curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64
2. chmod +x minikube-linux-amd64
3. sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64



4. minikube start – This will download the necessary pre-requisites before launching the environment.

```
┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0      0   0      0   0      0      0      0                              0
  0      0   0      0   0      0      0      0                              0
100 128.6M 100 128.6M   0      0  4.33M      0   00:29   00:29          4.35M

┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# chmod +x minikube-linux-amd64

┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64

┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# minikube start --force
😄  minikube v1.38.1 on Debian kali-rolling
❗  minikube skips various validations when --force is supplied; this may lead to unexpected behavior
✨  Automatically selected the docker driver. Other choices: none, ssh
🛑  The "docker" driver should not be used with root privileges. If you wish to continue as root, use --force.
💡  If you are running minikube within a VM, consider using --driver=none:
📘    https://minikube.sigs.k8s.io/docs/reference/drivers/none/
❗  Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
📌  Using Docker driver with root privileges
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.50 ...
💾  Downloading Kubernetes v1.35.1 preload ...
    > preloaded-images-k8s-v18-v1 ... :  272.45 MiB / 272.45 MiB  100.00% 3.12 Mi
    > gcr.io/k8s-minikube/kicbase ... :  519.58 MiB / 519.58 MiB  100.00% 4.04 Mi
🔥  Creating docker container (CPUs=2, Memory=3072MB) ...
🐳  Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔗  Configuring bridge CNI (Container Networking Interface) ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass
💡  kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─#
```

5. Alternatively, minikube can download the appropriate version of kubectl and you should be able to use it like this:

```
┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# minikube kubectl -- get po -A
NAMESPACE     NAME                               READY   STATUS    RESTARTS      AGE
kube-system   coredns-7d764666f9-5w46x           1/1     Running   0             2m31s
kube-system   etcd-minikube                      1/1     Running   0             2m39s
kube-system   kube-apiserver-minikube            1/1     Running   0             2m39s
kube-system   kube-controller-manager-minikube   1/1     Running   0             2m39s
kube-system   kube-proxy-k9ssz                   1/1     Running   0             2m32s
kube-system   kube-scheduler-minikube            1/1     Running   0             2m39s
kube-system   storage-provisioner                1/1     Running   1 (2m ago)    2m31s

┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─#
```

6. alias kubectl="minikube kubectl --"

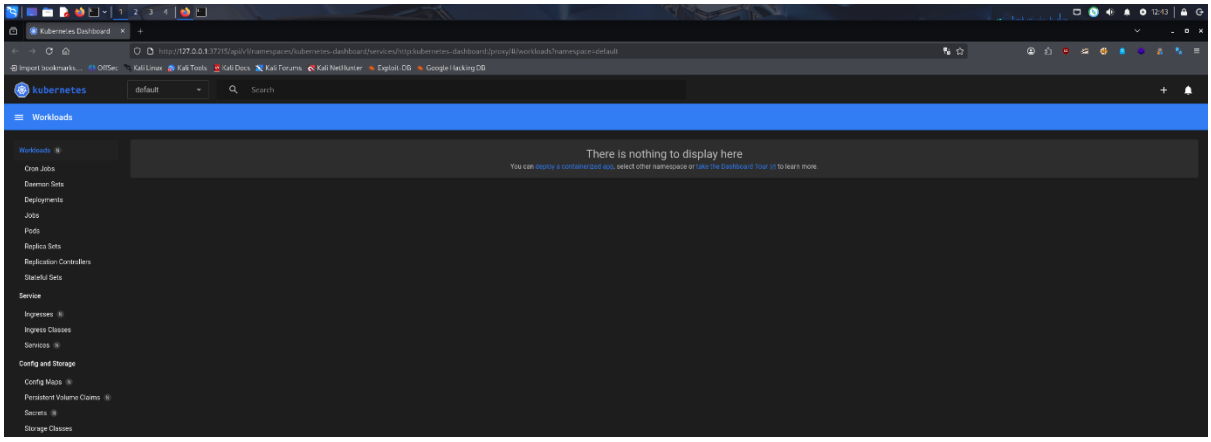
```
┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# alias kubectl="minikube kubectl --"

┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─#
```

To verify and to check the installation of minikube, run the following command.

Initially, some services such as the storage-provisioner, may not yet be in a Running state. This is a normal condition during cluster bring-up, and will resolve itself momentarily. For additional insight into your cluster state, minikube bundles the Kubernetes Dashboard, allowing you to get easily acclimated to your new environment:

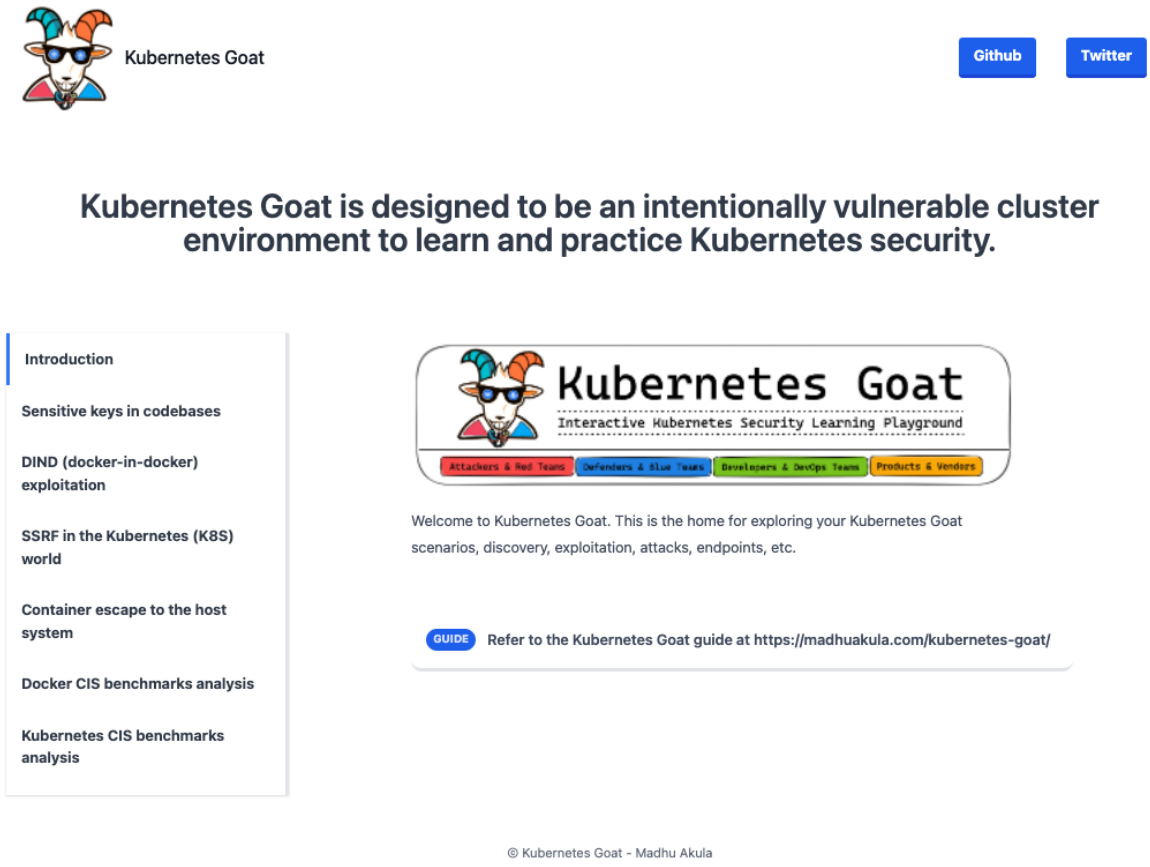
If everything is perfectly installed, the following dashboard will appear on the screen.



2. Setting up Kubernetes-Goat on the system.

What is Kubernetes Goat?

The Kubernetes Goat is designed to be an intentionally vulnerable cluster environment to learn and practice Kubernetes security



Steps to Install

Few Steps to Take care before Installing Kubernetes Goat:-

- Ensure you have the helm package manager installed.

curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4

chmod 700 get_helm.sh

./get_helm.sh

```
┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4

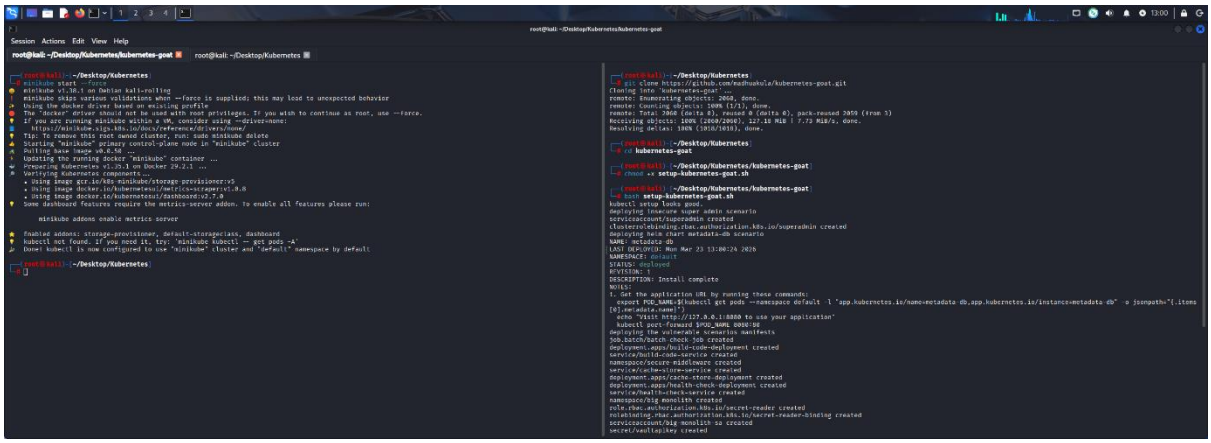
┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# chmod 700 get_helm.sh

┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# ./get_helm.sh
Downloading https://get.helm.sh/helm-v4.1.3-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm

┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─#
```

After Installing Helm, proceed ahead -

1. git clone https://github.com/madhuakula/kubernetes-goat.git
2. cd kubernetes-goat
3. chmod +x setup-kubernetes-goat.sh
4. bash setup-kubernetes-goat.sh



Ensure the pods are running before running the access script.

kubectl get pods

```
┌──(root㉿kali)-[~/Desktop/Kubernetes/kubernetes-goat]
└─# kubectl get pods
NAME                                      READY   STATUS    RESTARTS   AGE
batch-check-job-65rtt                     1/1     Running   0          6m3s
build-code-deployment-84fb858df9-pp7k9    1/1     Running   0          6m3s
health-check-deployment-b7f74cdc8-ktvz5   1/1     Running   0          6m3s
hidden-in-layers-mxlc6                    1/1     Running   0          6m1s
internal-proxy-deployment-c59c45d94-2zf5h 2/2     Running   0          6m2s
kubernetes-goat-home-deployment-cd46cd955-pw5bq   1/1     Running   0          6m2s
metadata-db-6fbb8cd6d7-t2x79              1/1     Running   0          6m4s
poor-registry-deployment-7b75fbd7-fh974   1/1     Running   0          6m2s
system-monitor-deployment-866f697c75-dq9gv        1/1     Running   0          6m1s

┌──(root㉿kali)-[~/Desktop/Kubernetes/kubernetes-goat]
└─#
```

Once all the pods start running, run the following command to boot up Kubernetes-goat.

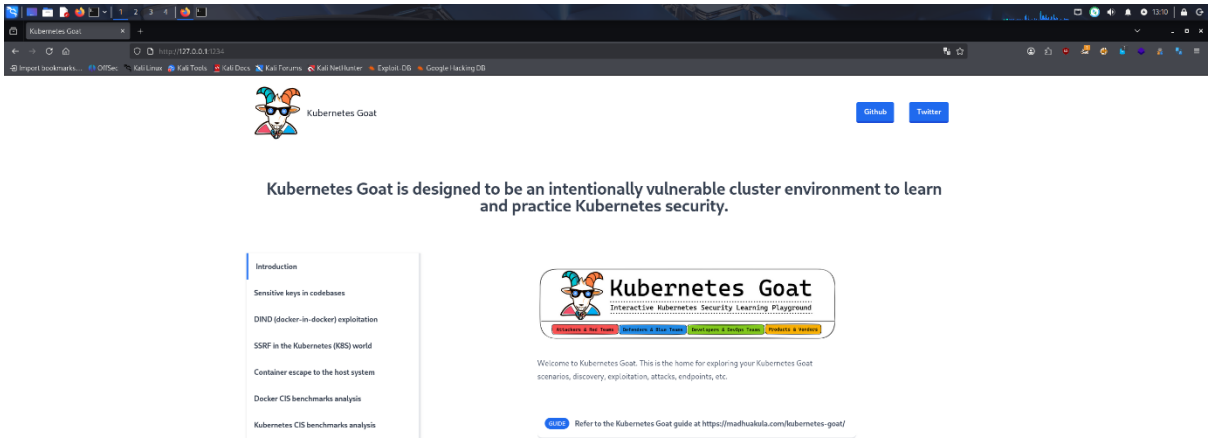
- bash access-kubernetes-goat.sh

```
┌──(root㉿kali)-[~/Desktop/Kubernetes/kubernetes-goat]
└─# bash access-kubernetes-goat.sh
kubectl setup looks good.
Creating port forward for all the Kubernetes Goat resources to locally. We will be using 1230 to 1236 ports locally!
Visit http://127.0.0.1:1234 to get started with your Kubernetes Goat hacking!

┌──(root㉿kali)-[~/Desktop/Kubernetes/kubernetes-goat]
```



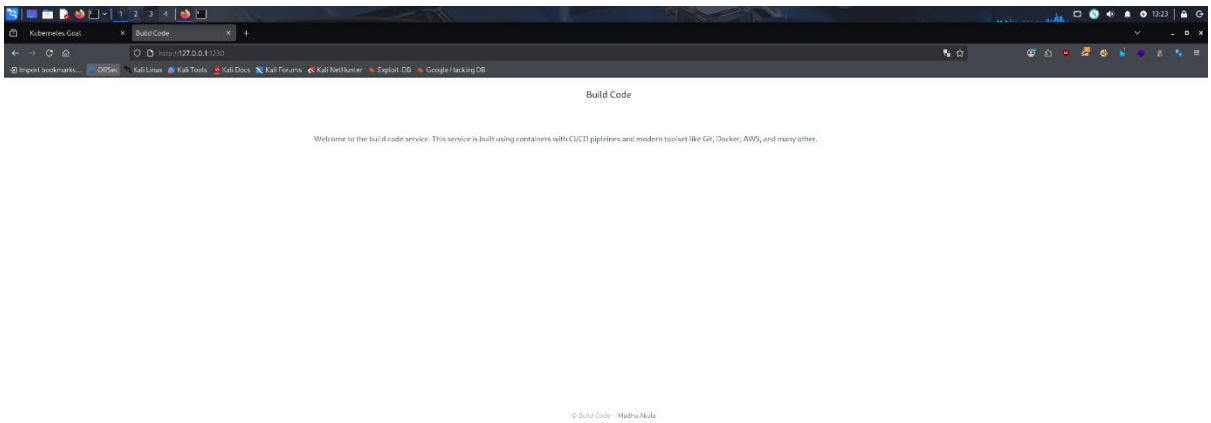
Congratulations!! Kubernetes Goat has been setup!!!!1



Solution Walkthrough

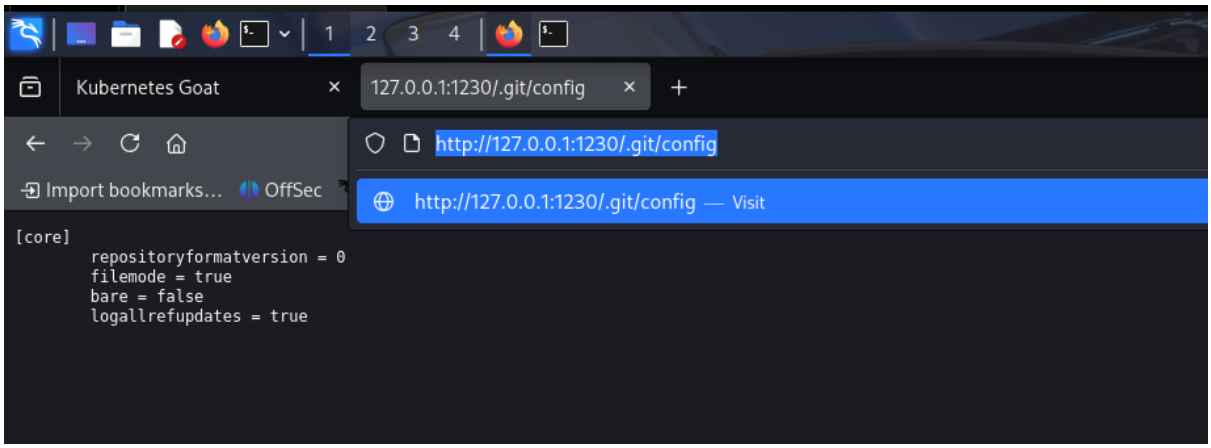
Scenario 1: Sensitive Keys in Codebases: - Developers tend to commit sensitive information to version control systems. As we are moving towards CI/CD and GitOps systems, we tend to forgot identifying sensitive information in code and commits. Let's see if we can find something cool here!

Vulnerability Explanation: Secrets committed to Git, exposed via .git in container. Allows repo dump, history scan for AWS keys/flag.



Solution:

Visit http://127.0.0.1:1230/.git/config. : Git repo exposed via web server misconfiguration



Since we already know via the hint that the code might be hosted over Git. Hence after enumeration I found that there exists a ./git folder. Hence, I downloaded the entire Git repository remotely



```
┌──(root㉿kali)-[~/Desktop/Kubernetes]
└─# cd gitdump

┌──(root㉿kali)-[~/Desktop/Kubernetes/gitdump]
└─# ls -al
total 11580
drwxr-xr-x 4 root root     4096 Mar 23 14:14 .
drwxr-xr-x 5 root root     4096 Mar 23 14:14 ..
-rwxr-xr-x 1 root root 11773672 Mar 23 14:14 app
drwxr-xr-x 7 root root     4096 Mar 23 14:14 .git
-rw-r--r-- 1 root root      105 Mar 23 14:14 go.mod
-rw-r--r-- 1 root root    49893 Mar 23 14:14 go.sum
-rw-r--r-- 1 root root      420 Mar 23 14:14 main.go
-rw-r--r-- 1 root root      180 Mar 23 14:14 README.md
drwxr-xr-x 2 root root     4096 Mar 23 14:14 views

┌──(root㉿kali)-[~/Desktop/Kubernetes/gitdump]
└─#
```

This shows that there might exists a git commit history in the repository downloaded locally. Hence, I started exploring the git repository.

```
┌──(root㉿kali)-[~/Desktop/Kubernetes/gitdump]
└─# git log
commit 905dcec070d86ce60822d790492d7237884df60a (HEAD → master)
Author: Madhu Akula <madhu.akula@hotmail.com>
Date:   Fri Nov 6 23:42:28 2020 +0100

    Final release

commit 3292ff3bd8d96f192a9d4eb665fdd1014d87d3df
Author: Madhu Akula <madhu.akula@hotmail.com>
Date:   Fri Nov 6 23:40:59 2020 +0100

    Updated the docs

commit 7daa5f4cda812faa9c62966ba57ee9047ee6b577
Author: Madhu Akula <madhu.akula@hotmail.com>
Date:   Fri Nov 6 23:39:21 2020 +0100

    updated the endpoints and routes

commit d7c173ad183c574109cd5c4c648ffe551755b576
Author: Madhu Akula <madhu.akula@hotmail.com>
Date:   Fri Nov 6 23:31:06 2020 +0100

    Inlcuded custom environmental variables

commit bb2967a6f26fb59bf64031bbb14b4f3e233944ca
Author: Madhu Akula <madhu.akula@hotmail.com>
Date:   Fri Nov 6 23:28:33 2020 +0100

    Added ping endpoint

commit 599f377bde4c3c5c8dc0d7700194b5b2b0643c0b
Author: Madhu Akula <madhu.akula@hotmail.com>
Date:   Fri Nov 6 23:24:56 2020 +0100

    Basic working go server with fiber

commit 4dc0726a546f59e0f4cda837a07032c62ee137bf
Author: Madhu Akula <madhu.akula@hotmail.com>
Date:   Fri Nov 6 23:21:48 2020 +0100

    Initial commit with README

┌──(root㉿kali)-[~/Desktop/Kubernetes/gitdump]
└─#
```

```
┌──(root㉿kali)-[~/Desktop/Kubernetes/gitdump]
└─# git checkout d7c173ad183c574109cd5c4c648ffe551755b576
Note: switching to 'd7c173ad183c574109cd5c4c648ffe551755b576'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at d7c173a Inlcuded custom environmental variables

┌──(root㉿kali)-[~/Desktop/Kubernetes/gitdump]
└─# cat .env
[build-code-aws]
aws_access_key_id = AKIVSHD6243H22G1KIDC
aws_secret_access_key = cgGn4+gDgnriogn4g+34ig4bg34g44gg4Dox7c1M
k8s_goat_flag = k8s-goat-51bc78332065561b0c99280f62510bcc
```

Here, we can observe that there are sensitive details disclosed such as access key and secret access key which can be used to access cloud environment.

Scenario 2: DIND (docker-in-docker) exploitation

Most of the CI/CD and pipeline system which use Docker and build containers for you with in the pipeline use something called DIND (docker-in-docker). Here in this scenario, we try to exploit and gain access to host system.

Vulnerability Explanation: